



COMPILADORES

PROJETO – Fase 2 – Análise Semântica e Geração de Código

OBJETIVO

A proposta desta atividade é a implementação das etapas de **análise semântica** e **geração de código intermediário** do compilador. A implementação dessa fase será baseada na implementação realizada na **fase 1**.

ANÁLISE SEMÂNTICA

Na **análise semântica** o seu compilador deverá verificar se as construções sintáticas da fase anterior estão coerentes.

O compilador implementado na fase anterior deve manter as funcionalidades de identificação de erros **léxicos** e **sintáticos**, e adicionalmente, emitir as mensagens de **erros semânticos**, caso ocorram.

O compilador fará somente a verificação semântica para variáveis em três momentos:

- **Declaração:** na seção de declaração de variável o compilador deve garantir que os identificadores usados no nome de variável sejam únicos, ou seja, não podendo ter duas variáveis declaradas com o mesmo nome. Caso aconteça uma repetição de identificador o **compilador deve ser finalizado informando que ocorreu um erro semântico** explicativo.
- **Corpo do programa:** as variáveis declaradas na seção de declaração de variáveis podem ser referenciadas nos comandos de atribuição, nas expressões e nas chamadas das funções de entrada e saída. Portanto toda vez que uma variável for referenciada no corpo de programa, o compilador deve verificar se a variável foi declarada corretamente, **caso não tenha sido declarada é gerado um erro semântico** explicativo e o compilador é finalizado.
- **Tipo de dados:** fazer a checagem de tipos dos identificadores utilizados em expressões de comandos de atribuição e em condições.

Para isso deverá ser implementada uma **minitabela de símbolos** que armazenará as variáveis declaradas (**identificador, endereço e tipo**). O endereço da variável deve ser a ordem em que a variável foi declarada, assim a primeira variável tem endereço **0**, a segunda endereço **1** e assim sucessivamente.

Para armazenar as informações referentes as variáveis, a **minitabela de símbolos** deverá, **obrigatoriamente**, ser implementada utilizando a estrutura de dados **lista encadeada**, cada nó da tabela de símbolos é do tipo TNo descrito abaixo:

```
typedef struct _TNo {  
    char ID [16];  
    int endereco;  
    char tipo [7];  
    struct _TNo *prox;
```



} TNo;

GERAÇÃO DE CÓDIGO INTERMEDIÁRIO

Implementar a **geração de código** para todos os comandos e procedimentos da linguagem, usando os padrões de código apropriados. Implementar e utilizar um gerador de rótulos. Criar um arquivo TXT contendo as instruções da máquina-objeto. As variáveis deverão ser referenciadas pelos seus nomes.

Nos exemplos de geração de código intermediário são apresentadas duas funções: **proximo_rotulo()** e **busca_tabela_simbolos()**, recomendamos fortemente que você implemente conforme proposto, pois tornam a implementação modularizada e a geração de código mais simples.

Função **proximo_rotulo()** - retorna o valor do próximo rótulo consecutivo positivo (por exemplo **1, 2, 3, ...**). **Importante:** Como todas as funções são recursivas, deve-se tomar o cuidado na atribuição das variáveis locais que irão receber o retorno da função e/ou a ordem de chamadas da função **proximo_rotulo()**.

Função **busca_tabela_simbolos()** - recebe como parâmetro o atributo **atributo_ID** do átomo corrente (um vetor de caracteres) e retorna o endereço da variável armazenado na minitabela de símbolos, caso o identificador **não conste da tabela de símbolos a função gera um erro semântico**, que é informado na tela do computador, em seguida o processo de compilação é finalizado.

Lembre-se que a variável **InfoAtomo** é uma variável global do tipo **TInfoAtomo** e é atualizada na função **consome()** e armazena os atributos do **átomo** reconhecido no **analisador léxico**. Sugestão: declare também o início da lista encadeada em uma variável global de forma a facilitar a implementação.

OBSERVAÇÕES IMPORTANTES

O programa deve estar bem documentado e pode ser feito em grupo de até **2 alunos**, não esqueçam de colocar o **nome dos integrantes** do grupo no arquivo fonte do trabalho.

Fica **terminantemente proibido** o uso de ferramentas de **Inteligência Artificial** para a **geração automática do código do projeto**. Qualquer tentativa de burlar esta restrição será considerada uma infração disciplinar, conforme o **COMUNICADO DA FCI de 05/02/2025**.

O trabalho será avaliado de acordo com os seguintes critérios:

- Funcionamento do programa, caso programa apresentarem **warning** ao serem compilados serão penalizados. Após a execução o programa deve finalizar com **retorno igual a 0**;
- Caso o **programa não compile ou não execute** será penalizado com a **NOTA 0.0**;
- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador do **MinGW** com **VSCode**, para configurar sua máquina no Windows acesse:

<https://www.doug.dev.br/2022/Instalacoes-e-configuracoes-para-programar-em-C-usando-o-VS-Code/>

- Compile seu programa com o seguinte comando abaixo, considere que o programa fonte do seu compilador seja compilador.c:

```
gcc -g -Og -Wall compilador.c -o compilador
```

- O quão fiel é o programa quanto à descrição do enunciado, principalmente ao formato do **arquivo de entrada**;



-
- Clareza e organização, programas com código confuso (linhas longas, variáveis com nomes não-significativos,) e desorganizado (sem indentação, sem comentários,) também serão levados em consideração na correção.
 - Entrega de um arquivo **Readme.txt** explicando até a parte do trabalho que foi feito.
 - Todos os grupos precisarão apresentar o projeto.